设计报告：基于电商数据的中国高频价格指数计算

一、项目背景与目标

传统消费者价格指数（CPI）依赖人工调研和月度统计，存在时效性差、颗粒度粗和覆盖不全等缺陷。随着电商数据的广泛生成，利用电商平台的高频（按日）价格和销量数据，构建实时、高频的价格指数成为可能。该指数能更灵敏地反映市场价格变动，辅助宏观经济分析和政策制定 7。

本项目旨在基于阿里云天池公开电商数据，利用云服务和大数据技术，实现一套自动化、高并发、可复用的高频价格指数计算体系，具体目标包括：

- 掌握对象存储（OSS）数据管理及权限配置
- 利用 ClickHouse 实现高效列式查询与价格指数构建
- 基于 Python 实现自动化计算流程
- （可选）使用数据湖方案优化存储与计算架构
- 实现日度价格指数趋势可视化

二、数据获取与预处理

数据是价格指数计算的基础，质量直接影响指数的准确性和可靠性。电商平台产生的数据量大且更新快，利用云对象存储（OSS）能高效管理这些海量数据。将原始 CSV 文件上传 OSS 后，借助 ClickHouse 的 OSS 表引擎直接访问数据，避免重复导入和存储，提升数据处理效率。

数据预处理的核心是统一格式和清洗异常数据。时间格式统一保证后续按日聚合的准确性；价格单位统一避免计算误差；分类编码规范化便于分类汇总。异常数据（如价格为空、价格为零或负数、销量异常）会导致指数失真，必须过滤或修正。此阶段的目标是为后续计算提供干净、标准化的数据输入。

1. 数据源
   - **category.csv**：商品分类信息
   - **price.csv**：商品每日价格与销量数据

数据来自阿里云天池平台公开数据集，上传至对象存储 OSS，创建个人 Bucket 并配置访问权限。

2. 数据上传与映射
   - 将 CSV 文件上传至 OSS Bucket。
   - 在 ClickHouse 中使用 OSS 表引擎创建外部表，实现数据映射和访问，避免重复导入，支持实时查询。

示例 ClickHouse 建表（简化）：

sql

```sql
CREATE TABLE price_oss (
    item_id String,
    category_id String,
    date Date,
    price Float64,
    sales UInt32
) ENGINE = OSS('oss://bucket-name/price.csv', 'CSV');
```

3. 数据清洗与字段适配
   - 统一时间格式（如转换为 YYYY-MM-DD）

- 价格单位统一（如元）
- 分类编码标准化
- 处理缺失值和异常值（如价格为空、销量异常）
- 删除重复数据

Python 示例代码：

python

```python
import pandas as pd

# 读取数据
df = pd.read_csv('price.csv')

# 统一时间格式
df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d')

# 价格缺失处理
df = df.dropna(subset=['price'])

# 异常价格过滤（如价格<=0）
df = df[df['price'] > 0]

# 重置索引
df.reset_index(drop=True, inplace=True)
```

三、架构设计

1. 存储层
   - **对象存储（OSS）**：存储原始 CSV 数据，支持弹性扩展和高并发访问。
   - **ClickHouse**：列式数据库，支持高效的列查询和聚合计算，使用 OSS 表引擎直接访问 OSS 数据。
2. 计算层
   - **ClickHouse SQL**：实现价格指数的核心计算，包括异常处理和加权平均。
   - **Python 自动化脚本**：调度任务，调用 ClickHouse 接口，完成数据预处理、计算和结果导出。
   - （可选）数据湖方案如阿里云 Data Lake Analytics，实现更灵活的计算。
3. 可视化层
   - 使用阿里云 Quick BI 或 Python 的 matplotlib/seaborn 进行指数趋势的可视化展示。
   - 输出日度价格指数趋势图，支持交互式分析。

四、价格指数核心算法设计

价格指数反映商品价格随时间的变动情况，核心是计算不同类别商品的加权平均价格，并构建时间序列。

- **异常处理**：在 ClickHouse 中利用 SQL 的过滤条件，剔除价格为空、价格非正、销量异常等数据，确保计算基础数据的准确性。
- **加权平均价格**：销量作为权重，反映市场实际交易规模，避免少量高价商品对指数产生过大影响。加权平均价格更能真实反映消费者实际支付水平。

- **链式与固定基期指数**：固定基期指数以某一基准日价格为参照，计算后续日价格相对基期的变化；链式指数则通过连续日期间价格比率累积，适应价格波动较大的情况。两者结合可满足不同分析需求。
- **多级分类指数计算**：先计算最细分类的价格指数，再根据动态或固定权重汇总到上级类别，最终形成整体价格指数。动态权重根据销量变化调整，反映市场结构变化；固定权重保证大类结构稳定，避免权重流失。

1. 异常处理机制（ClickHouse 实现）
   - 过滤价格为空或非正数的记录
   - 过滤销量异常（如负数或异常大）
   - 采用 SQL 中的 WHERE 条件和 IF 函数实现异常过滤

示例 SQL：

```
SELECT
    category_id,
    date,
    AVGIf(price, price > 0) AS avg_price,
    SUMIf(sales, price > 0) AS total_sales
FROM price_oss
WHERE price IS NOT NULL AND price > 0 AND sales >= 0
GROUP BY category_id, date
```

2. 价格指数计算
   - **分类价格指数**：对每个类别，按销量加权平均价格计算日均价格。

计算公式：

Pc,t=∑i∈cpi,t×si,t∑i∈csi,t$P_{c,t}=\frac{\sum_{i\in cpi,t}\times si,t}{\sum_{i\in csi,t}}$

其中，pi,t$p_{i,t}$ 为商品 i 在日期 t 的价格，si,t$s_{i,t}$ 为销量。

- **链式指数与固定基期指数**
    - 固定基期指数：选定基期日 t0t_0，计算各日价格相对基期价格的比值。

Ic,t=Pc,tPc,t0×100$I_{c,t}=\frac{P_{c,t}}{P_{c,t_0}}\times 100$

    - 链式指数：连续计算相邻两日价格比率，逐步累积。
- **整体指数**：对所有类别指数按权重（如类别销售额占比）加权平均。

示例 ClickHouse SQL 计算链式指数：

sql

```
WITH daily_avg AS (
    SELECT
        category_id,
        date,
        SUM(price * sales) / SUM(sales) AS weighted_price
    FROM price_oss
    WHERE price > 0 AND sales >= 0
    GROUP BY category_id, date
),
base_prices AS (
    SELECT category_id, weighted_price AS base_price
    FROM daily_avg
```

```
    WHERE date = '2023-01-01'   -- 基期日期
)
SELECT
    d.category_id,
    d.date,
    d.weighted_price / b.base_price * 100 AS price_index
FROM daily_avg d
JOIN base_prices b ON d.category_id = b.category_id
ORDER BY d.category_id, d.date
```

五、自动化计算流程（Python 示例）

自动化脚本负责整个数据处理和指数计算过程的调度，实现高效、稳定的运行。

- 通过 ClickHouse 客户端接口执行 SQL 查询，完成数据预处理和价格指数计算。
- 将结果导出为 DataFrame 格式，方便后续分析和可视化。
- 利用 Python 绘图库绘制日度价格指数趋势图，支持多类别对比分析。
- 脚本支持参数化配置（如基期日期、类别筛选），方便复用和扩展。
- 定时调度结合云服务（如阿里云函数计算或调度服务）实现每日自动更新，保证指数的高频实时性。

```
import clickhouse_connect
import pandas as pd
import matplotlib.pyplot as plt

# 连接 ClickHouse
client = clickhouse_connect.get_client(host='host', port=9000, username='user', password='pass')

# 执行价格指数计算 SQL
query = """
WITH daily_avg AS (
    SELECT
        category_id,
        date,
        SUM(price * sales) / SUM(sales) AS weighted_price
    FROM price_oss
    WHERE price > 0 AND sales >= 0
    GROUP BY category_id, date
),
base_prices AS (
    SELECT category_id, weighted_price AS base_price
    FROM daily_avg
    WHERE date = '2023-01-01'
)
SELECT
```

```
    d.category_id,
    d.date,
    d.weighted_price / b.base_price * 100 AS price_index
FROM daily_avg d
JOIN base_prices b ON d.category_id = b.category_id
ORDER BY d.category_id, d.date
"""

df_index = client.query_dataframe(query)

# 可视化某类别价格指数趋势
category_to_plot = '食品'
df_cat = df_index[df_index['category_id'] == category_to_plot]
plt.plot(df_cat['date'], df_cat['price_index'])
plt.title(f'{category_to_plot} 日度价格指数趋势')
plt.xlabel('日期')
plt.ylabel('价格指数')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

六、测试与验证

1. 单元测试
   - 构造小规模 CSV 数据，上传 OSS，测试字段解析、时间格式转换、异常过滤。
   - 验证 ClickHouse SQL 输出结果正确。
2. 集成测试
   - 验证商品覆盖率≥80%，确保指数代表性。
   - 异常数据过滤准确率≥95%。
   - 使用历史数据与官方 CPI 对比，验证指数趋势合理性。